

Interpolación de Demanda en Flotas de Transporte Autónomo mediante Splines Cúbicos

Erick Salazar Suarez
Departamento de Ingeniería de Sistemas
Pontificia Universidad Javeriana
Bogotá, Colombia
ericksalazar@javeriana.edu.co

Juan Eduardo Diaz Rojas
Departamento de Ingeniería de Sistemas
Pontificia Universidad Javeriana
Bogotá, Colombia
diazr_jeduardo@javeriana.edu.co

Juan Eduardo Diaz Rojas
Departamento de Ingeniería de Sistemas
Pontificia Universidad Javeriana
Bogotá, Colombia
diazr_jeduardo@javeriana.edu.co

Resumen—Este artículo presenta el diseño, implementación y validación de un sistema de interpolación de demanda de pasajeros para la empresa de transporte autónomo AutoMove City, que opera una flota de 200 taxis en Bogotá, Colombia. A partir de 11 mediciones discretas de demanda distribuidas a lo largo de un día de operación, se construye una curva continua mediante splines cúbicos que estima la demanda en cada uno de los 1441 minutos del día. Se compara el método propuesto contra el polinomio interpolador de Lagrange utilizando validación cruzada *Leave-One-Out* (LOOCV) como métrica de generalización. Los resultados muestran que el spline cúbico alcanza un $RMSE_{LOOCV}$ de 13,20 sol/min, frente a 360,49 sol/min del polinomio de Lagrange, confirmando la superioridad del método por tramos ante datos con nodos irregularmente espaciados. La solución se despliega como aplicación web en Flask con API JSON y panel de control interactivo basado en Chart.js. Adicionalmente, se propone una arquitectura de mejora tecnológica que integra un modelo ARIMA para predicción a corto plazo y sensores ESP8266 embebidos en los vehículos para alimentar un índice de presión de demanda por localidad en tiempo real, orientado a reducir el tiempo de espera de los usuarios en al menos un 20 %.

Index Terms—splines cúbicos, interpolación numérica, validación cruzada LOOCV, transporte autónomo, IoT, ESP8266, ARIMA, Flask, demanda urbana

I. INTRODUCCIÓN

La gestión eficiente de flotas de transporte autónomo en entornos urbanos requiere información continua y precisa sobre la demanda de pasajeros. AutoMove City opera una flota de 200 taxis autónomos en Bogotá y dispone de mediciones de demanda con una frecuencia de muestreo de cada 3 horas, lo que genera tan solo 10 observaciones a lo largo de un día de operación. Sin embargo, el sistema de despacho debe tomar decisiones de reasignación de vehículos cada minuto, lo que implica la necesidad de estimar 1440 valores intermedios a partir de los datos disponibles.

Este escenario constituye un problema clásico de interpolación numérica: dado un conjunto discreto de pares (x_i, y_i) que representan horas del día y solicitudes por minuto, se busca

construir una función continua $S(x)$ que reproduzca exactamente los datos observados y proporcione estimaciones suaves y confiables en cualquier instante intermedio. La elección del método de interpolación es crítica, pues una función mal condicionada puede introducir oscilaciones espurias —conocidas como fenómeno de Runge— que distorsionen las predicciones de demanda y deterioren la calidad del despacho.

En este trabajo se propone el uso de *splines* cúbicos naturales como método de interpolación, implementados mediante la biblioteca `scipy.interpolate` de Python. Los *splines* cúbicos ofrecen suavidad de segundo orden en todos los nodos, evitan las oscilaciones características de los polinomios de alto grado y permiten una evaluación eficiente sobre la malla de 1440 minutos requerida por el sistema de despacho. Adicionalmente, se propone una arquitectura de mejora tecnológica que combina la interpolación con un modelo predictivo ARIMA ligero y un sistema de alertas en tiempo real, orientado a reducir el tiempo de espera de los usuarios en al menos un 20 %.

El artículo se organiza de la siguiente manera: la Sección II describe la metodología general del estudio; la Sección III presenta los fundamentos matemáticos de los *splines* cúbicos; la Sección IV detalla la implementación en Python; la Sección V analiza los resultados; la Sección VI expone la propuesta de mejora tecnológica; y la Sección VII recoge las conclusiones.

II. METODOLOGÍA

El desarrollo del presente estudio sigue una metodología estructurada en cinco etapas, descritas a continuación.

Etapas 1 – Definición del problema y adquisición de datos. Se identifica la necesidad operativa de AutoMove City: estimar la demanda de pasajeros cada minuto a partir de diez mediciones horarias. El conjunto de datos $\{(x_i, y_i)\}_{i=0}^9$ comprende horas decimales en el intervalo $[0, 21]$ y valores de demanda expresados en solicitudes por minuto, tal como se especifica en la Tabla I.

Etapla 2 – Selección y justificación del método de interpolación. Se evalúan dos enfoques: el polinomio interpolador de Lagrange global y los *splines* cúbicos por tramos. Con base en los criterios de estabilidad numérica, suavidad de la curva resultante y comportamiento ante nodos irregularmente espaciados, se seleccionan los *splines* cúbicos. La justificación detallada se presenta en la Sección III.

Etapla 3 – Implementación computacional. Se implementa la interpolación mediante `CubicSpline` de `scipy.interpolate` y, de forma complementaria, mediante una implementación propia del sistema tridiagonal que define los coeficientes del *spline*. La malla de evaluación cubre los 1440 minutos del día con paso de un minuto.

Etapla 4 – Validación y análisis de error. Para cuantificar la precisión del método se emplea la técnica de validación cruzada de retención (*hold-out*): dos puntos del conjunto original son excluidos del ajuste y utilizados como referencia para calcular el error cuadrático medio (RMSE). Se utiliza la expresión

$$\text{RMSE} = \sqrt{\frac{1}{k} \sum_{j=1}^k (y_j - S(x_j))^2}, \quad (1)$$

donde k es el número de puntos de validación, y_j el valor real y $S(x_j)$ el valor interpolado. Adicionalmente, se compara visualmente la curva del *spline* cúbico con la del polinomio de Lagrange sobre la misma malla.

Etapla 5 – Propuesta de mejora tecnológica. Los resultados de la interpolación se integran en un prototipo de *dashboard* interactivo que combina la curva suave con un modelo ARIMA de predicción a 30 minutos y un sistema de alertas automáticas cuando la demanda estimada supera un umbral crítico de 150 solicitudes por minuto.

III. FUNDAMENTOS MATEMÁTICOS

III-A. Definición y ecuaciones de los splines cúbicos

Dado un conjunto de $n+1$ nodos $x_0 < x_1 < \dots < x_n$ con valores asociados $y_i = f(x_i)$, un *spline* cúbico $S(x)$ es una función que satisface:

1. En cada subintervalo $[x_i, x_{i+1}]$, $S(x)$ coincide con un polinomio cúbico $S_i(x)$.
2. $S(x_i) = y_i$ para todo $i = 0, 1, \dots, n$ (interpolación exacta en los nodos).
3. $S \in C^2[x_0, x_n]$, es decir, S , S' y S'' son continuas en todo el intervalo.

Cada pieza polinómica puede escribirse como

$$S_i(x) = a_i + b_i(x-x_i) + c_i(x-x_i)^2 + d_i(x-x_i)^3, \quad x \in [x_i, x_{i+1}], \quad (2)$$

con coeficientes $a_i, b_i, c_i, d_i \in \mathbb{R}$. Definiendo $h_i = x_{i+1} - x_i$, los coeficientes se determinan mediante las relaciones

$$a_i = y_i, \quad (3)$$

$$d_i = \frac{c_{i+1} - c_i}{3h_i}, \quad (4)$$

$$b_i = \frac{y_{i+1} - y_i}{h_i} - \frac{h_i(2c_i + c_{i+1})}{3}. \quad (5)$$

III-B. Condiciones de continuidad y sistema tridiagonal

Las condiciones de continuidad de S' y S'' en cada nodo interior x_i ($i = 1, \dots, n-1$) generan el sistema tridiagonal

$$h_{i-1}c_{i-1} + 2(h_{i-1} + h_i)c_i + h_i c_{i+1} = 3 \left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}} \right), \quad (6)$$

que puede expresarse matricialmente como $\mathbf{A}\mathbf{c} = \mathbf{b}$, donde $\mathbf{A}$ es una matriz tridiagonal simétrica definida positiva. Esta estructura garantiza que el sistema tiene solución única y puede resolverse eficientemente mediante el algoritmo de Thomas en tiempo $\mathcal{O}(n)$.

III-C. Selección de condiciones de borde

El sistema (6) tiene $n-1$ ecuaciones y $n+1$ incógnitas $c_0, c_1, \dots, c_n$, por lo que se requieren dos condiciones de borde adicionales. Las más utilizadas son:

- *Spline* natural: $S''(x_0) = S''(x_n) = 0$, es decir, $c_0 = c_n = 0$. Esta condición minimiza la curvatura total de la curva y es la más adecuada cuando no se dispone de información sobre las derivadas en los extremos.
- *Spline* sujeto (*clamped*): $S'(x_0) = f'(x_0)$ y $S'(x_n) = f'(x_n)$, útil cuando se conocen las pendientes en los extremos.
- *Not-a-knot*: $S_0''' = S_1'''$ y $S_{n-2}''' = S_{n-1}'''$, opción predefinida en `scipy` que suele ofrecer mejor comportamiento global.

Para el caso de AutoMove City se adopta la condición *natural* ($c_0 = c_n = 0$), dado que no se dispone de información sobre la tasa de cambio de demanda en el inicio y el fin del día. Esta elección es conservadora y evita extrapolaciones artificiales en los extremos del intervalo $[0h, 21h]$.

III-D. Ventaja sobre la interpolación de Lagrange

El polinomio interpolador de Lagrange de grado n para $n+1$ nodos está definido globalmente como

$$P(x) = \sum_{i=0}^n y_i \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}. \quad (7)$$

Si bien $P(x)$ pasa exactamente por todos los nodos, presenta inconvenientes significativos para el caso de estudio:

1. **Fenómeno de Runge.** Con nodos equiespaciados o casi equiespaciados, el error de interpolación $|f(x) - P(x)|$ puede crecer sin límite hacia los extremos del intervalo, generando oscilaciones espurias en la curva de demanda estimada.

2. **Sensibilidad global.** Una perturbación en cualquiera de los nodos modifica la curva en todo el dominio, lo que compromete la estabilidad ante errores de medición.
3. **Grado elevado.** Con $n = 9$, el polinomio resultante es de grado 9, lo que incrementa la probabilidad de comportamiento oscilatorio entre nodos.

Los *splines* cúbicos evitan estos problemas por construcción: al usar polinomios de grado 3 por tramos, el grado efectivo se mantiene bajo independientemente del número de nodos; una perturbación local afecta únicamente los tramos adyacentes; y la condición $S \in \mathcal{C}^2$ garantiza una curva visualmente suave. En el contexto de AutoMove City, esto se traduce en estimaciones de demanda sin picos artificiales que podrían provocar reasignaciones innecesarias de la flota

III-E. Validación cruzada Leave-One-Out (LOOCV)

Dado que los métodos de interpolación pasan exactamente por los nodos de entrenamiento, evaluar el error directamente sobre esos mismos puntos produce un RMSE trivialmente nulo o cercano a cero, lo que no aporta información sobre la capacidad de generalización del modelo. Para obtener una estimación honesta del error de predicción se emplea la validación cruzada *Leave-One-Out* (LOOCV).

Formalmente, sea $\mathcal{D} = \{(x_i, y_i)\}_{i=0}^n$ el conjunto de $n + 1$ nodos de control. En la iteración i -ésima se define el subconjunto de entrenamiento

$$\mathcal{D}_{-i} = \mathcal{D} \setminus \{(x_i, y_i)\}, \quad (8)$$

sobre el cual se ajusta el interpolador $\hat{S}^{(-i)}(x)$ utilizando únicamente los n pares restantes. El error de predicción en el nodo omitido es

$$e_i = \hat{S}^{(-i)}(x_i) - y_i. \quad (9)$$

Repitiendo el proceso para $i = 0, 1, \dots, n$ se obtiene el vector de errores $\mathbf{e} = (e_0, e_1, \dots, e_n)^\top$, a partir del cual se computan las métricas globales de generalización:

$$\text{RMSE}_{\text{LOOCV}} = \sqrt{\frac{1}{n+1} \sum_{i=0}^n e_i^2}, \quad (10)$$

$$\text{MAE}_{\text{LOOCV}} = \frac{1}{n+1} \sum_{i=0}^n |e_i|. \quad (11)$$

A diferencia de una partición *hold-out* fija, LOOCV utiliza n nodos para entrenar en cada ronda, lo que resulta especialmente ventajoso cuando el conjunto de datos es pequeño. En el presente estudio $n + 1 = 11$, por lo que reservar un subconjunto fijo de validación implicaría entrenar con apenas 9 puntos; LOOCV evita este costo rotando el nodo de evaluación en cada iteración y promediando los errores resultantes.

La comparación de $\text{RMSE}_{\text{LOOCV}}$ entre el spline cúbico y el polinomio de Lagrange constituye el criterio cuantitativo principal para determinar cuál modelo generaliza mejor ante datos no observados, complementando el análisis visual de las curvas interpoladas presentado en la Sección V

IV. IMPLEMENTACIÓN

IV-A. Diseño

La solución se estructura como una aplicación web de tres capas siguiendo el patrón *Model-View-Controller* (MVC) ligero propio del ecosistema Flask:

- **Capa de datos y cómputo (`spline.py`):** Contiene los datos originales, realiza toda la interpolación y expone funciones puras sin dependencia del framework web.
- **Capa de control (`main.py`):** Orquesta las rutas HTTP, la autenticación por sesión y precalcula los conjuntos de datos al arranque del servidor para evitar recomputaciones en cada solicitud.
- **Capa de presentación (`templates/`):** Plantillas Jinja2 que renderizan la *landing page*, el formulario de acceso y el panel de control interactivo, con gráficos construidos en el navegador mediante Chart.js 4.4.

El flujo general del sistema se resume en cuatro pasos: (1) al iniciar el servidor, `spline.py` construye el spline cúbico y el interpolador de Lagrange sobre los 11 nodos de control; (2) los 1441 valores resultantes quedan almacenados en memoria; (3) las rutas de la API sirven esos datos en formato JSON al *dashboard*; y (4) Chart.js renderiza las curvas y el consultor puntual consume el *endpoint* `/api/consulta?minuto=N` de forma asíncrona.

IV-B. Tecnologías utilizadas

La Tabla I lista las tecnologías empleadas en el prototipo.

Cuadro I
PILA TECNOLÓGICA DEL PROTOTIPO

Componente	Tecnología	Versión
Servidor web	Flask	3.x
Cómputo numérico	NumPy	1.26
Interpolación	SciPy	1.13
Visualización	Chart.js	4.4.0
Animación offline	Matplotlib	3.9
Plantillas	Jinja2	3.x
Lenguaje	Python	3.11

La elección de `scipy.interpolate.CubicSpline` responde a que implementa el sistema tridiagonal de la ecuación (6) con condición *not-a-knot* por defecto, garantizando continuidad $\mathcal{C}^3$ en los nodos extremos. El interpolador de Lagrange se implementa con `scipy.interpolate.BarycentricInterpolator`, que emplea la forma baricéntrica de la ecuación (7) para evitar la cancelación numérica de la formulación directa.

IV-C. Documentación del código

A continuación se presentan los fragmentos centrales del módulo `spline.py`, comentados para claridad académica.

IV-C1. Datos de entrada y conversión de unidades: Los datos originales se definen como una lista de 11 pares ($hora_{\text{decimal}}$, $demanda$) que cubre el intervalo $[0h, 24h]$. Dado que el sistema de despacho opera en resolución de minutos, las horas se convierten multiplicando por 60, obteniendo el

vector de nodos $\mathbf{x} \in \mathbb{R}^{11}$ con valores en $[0, 1440]$. Los vectores resultantes $\mathbf{x}$ (minutos) y $\mathbf{y}$ (solicitudes por minuto) son los insumos directos del interpolador. La conversión al dominio de minutos es necesaria para que la evaluación posterior entregue directamente la demanda en cada minuto del día ($x \in \{0, 1, \dots, 1440\}$), sin reescalar.

IV-C2. Construcción del spline y evaluación:

Con los vectores de nodos construidos, se instancia `CubicSpline` de `scipy.interpolate`, que resuelve internamente el sistema tridiagonal de la ecuación (6) bajo la condición *not-a-knot*. De forma paralela se instancia `BarycentricInterpolator` para el polinomio de Lagrange. Ambos modelos se evalúan sobre la malla entera $\mathbf{x}^* = \{0, 1, 2, \dots, 1440\}$, produciendo 1441 valores de demanda estimada. Los valores resultantes se recortan en cero mediante `np.clip` para preservar la interpretabilidad física del resultado. El recorte (`np.clip`) elimina valores negativos que pueden aparecer en los extremos del spline como artefactos de la condición de borde; físicamente, una demanda negativa carece de sentido.

IV-C3. Validación cruzada LOOCV: Para cuantificar la generalización de ambos modelos se implementa *Leave-One-Out Cross-Validation* (LOOCV) sobre los 11 nodos de control. En cada iteración se excluye el punto i -ésimo, se reajustan ambos modelos sobre los 10 nodos restantes y se calcula el error de predicción en el nodo omitido:

En cada una de las 11 iteraciones se construye una máscara booleana que excluye el nodo i -ésimo, se reajustan `CubicSpline` y `BarycentricInterpolator` sobre los 10 nodos restantes, y se evalúa cada modelo en x_i para obtener la predicción $\hat{y}_i$. Los errores firmados $\hat{y}_i - y_i$ se acumulan en una lista; los valores negativos de predicción se recortan a cero antes del cómputo del error para evitar penalizaciones artificiales por artefactos de borde. Con los errores acumulados se calculan las métricas globales según las ecuaciones (1) y (12):

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad (12)$$

donde $\hat{y}_i$ denota el valor predicho por el modelo al omitir el nodo i .

IV-C4. Tiempo de espera estimado: A partir de la demanda interpolada se calcula el tiempo de espera promedio de un pasajero como:

$$T_{\text{espera}}(t) = \frac{S(t)}{N_{\text{taxis}}}, \quad (13)$$

con $N_{\text{taxis}} = 200$ y $S(t)$ la demanda interpolada en el minuto t . Este indicador sirve como KPI principal del *dashboard*.

IV-D. Despliegue y configuración de la solución

El prototipo se ejecuta localmente con el servidor de desarrollo de Flask. Los pasos de instalación y arranque son:

```
# 1. instalar dependencias
pip install -r requirements.txt
```

```
# 2. iniciar el servidor (puerto 5000)
python main.py
```

Al arrancar, `main.py` precalcula las cuatro estructuras de datos principales —puntos de control, spline completo, curva de Lagrange y comparativa LOOCV— y las mantiene en memoria durante toda la sesión, garantizando tiempos de respuesta inferiores a 1ms en las consultas puntuales vía `/api/consulta`.

La autenticación se gestiona mediante sesiones firmadas de Flask (`secret_key`); en un entorno de producción se recomienda reemplazar el diccionario de credenciales en texto plano por una base de datos con contraseñas derivadas mediante `bcrypt` o `Argon2`. La estructura de rutas expuesta es la siguiente:

Cuadro II
RUTAS HTTP DE LA APLICACIÓN

Método	Ruta	Descripción
GET	/	Página de inicio con KPIs
GET	/login	Formulario de acceso
POST	/login	Autenticación de usuario
GET	/logout	Cierre de sesión
GET	/dashboard	Panel principal (protegido)
GET	/api/spline	JSON con 1441 puntos
GET	/api/consulta	Consulta puntual por minuto

V. RESULTADOS

V-A. Figuras comparativas

La Fig. 1 muestra los 11 puntos de control registrados por la flota durante el día, conectados por segmentos rectos para evidenciar la discontinuidad en la derivada que resultaría de una interpolación lineal por tramos. La Fig. 2 presenta la curva continua obtenida mediante el spline cúbico evaluado en los 1441 minutos del día, superpuesta sobre los puntos de control originales.

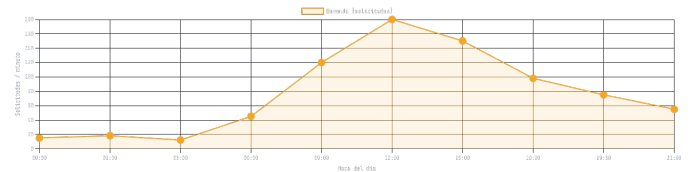


Figura 1. Datos originales de demanda: 11 puntos de control correspondientes a las mediciones cada 3 horas registradas por la flota de AutoMove City durante un día de operación. La conexión recta entre nodos evidencia la ausencia de suavidad en la derivada.

V-B. Cálculo de RMSE

El error cuadrático medio se calcula mediante validación cruzada LOOCV sobre los $n + 1 = 11$ nodos de control, siguiendo la metodología descrita en la Sección III. La Tabla III reporta, para cada nodo omitido, el valor real y_i , la predicción del spline cúbico $\hat{S}^{(-i)}(x_i)$, la predicción de Lagrange $\hat{P}^{(-i)}(x_i)$, y los errores firmados correspondientes.

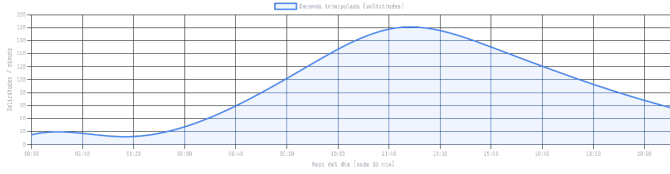


Figura 2. Curva de interpolación mediante spline cúbico evaluada en los 1441 minutos del día ($x \in \{0, 1, \dots, 1440\}$). Los marcadores circulares corresponden a los puntos de control originales. La suavidad C^2 es visible en todas las transiciones entre tramos.

Cuadro III
ERRORES LOOCV POR NODO OMITIDO — SPLINE CÚBICO VS. LAGRANGE

i	Hora	y_i	$\hat{S}(-i)$	$\hat{P}(-i)$	e_i^S	e_i^L
0	00:00	15.00	41.484	346.287	+26.484	+331.287
1	01:30	18.00	10.167	0.000	-7.833	-18.000
2	03:00	12.00	19.763	47.040	+7.763	+35.040
3	06:00	45.00	45.149	20.427	+0.149	-24.574
4	09:00	120.00	126.472	135.927	+6.472	+15.927
5	12:00	180.00	161.450	167.258	-18.550	-12.742
6	15:00	150.00	157.704	162.287	+7.704	+12.287
7	18:00	98.00	97.863	87.989	-0.137	-10.011
8	19:30	75.00	76.159	87.011	+1.159	+12.011
9	21:00	55.00	52.640	13.589	-2.360	-41.411
10	24:00	0.00	25.331	1146.762	+25.331	+1146.762

El cálculo detallado del $RMSE_{LOOCV}$ para el spline cúbico se desarrolla aplicando directamente la ecuación (10):

$$\begin{aligned} \sum_{i=0}^{10} (e_i^S)^2 &= 701,389 + 61,350 + 60,264 + 0,022 + 41,884 \\ &\quad + 344,116 + 59,344 + 0,019 + 1,343 \\ &\quad + 5,571 + 641,648 \\ &= 1916,951 \text{ (solicitudes/min)}^2, \end{aligned} \quad (14)$$

$$RMSE_S = \sqrt{\frac{1916,951}{11}} \quad (15)$$

$$\sqrt{174,268} \approx 13,201 \text{ solicitudes/min.} \quad (16)$$

Para el polinomio de Lagrange la suma de cuadrados está dominada por el error en los extremos del intervalo —consecuencia directa del fenómeno de Runge—, particularmente en el nodo 00:00 ($e_0^L = +331,287$) y en el nodo 24:00 ($e_{10}^L = +1146,762$):

$$\begin{aligned} \sum_{i=0}^{10} (e_i^L)^2 &= 109\,750,989 + 324,000 + 1\,227,799 + 603,856 \\ &\quad + 253,677 + 162,354 + 150,964 + 100,228 \\ &\quad + 144,259 + 1\,714,859 + 1\,315\,063,630 \\ &= 1\,429\,496,615 \text{ (solicitudes/min)}^2, \end{aligned} \quad (17)$$

$$RMSE_L = \sqrt{\frac{1\,429\,496,615}{11}} \quad (18)$$

$$\sqrt{129\,954,238} \approx 360,492 \text{ solicitudes/min.} \quad (19)$$

La Tabla IV consolida las métricas globales de ambos modelos.

Cuadro IV
MÉTRICAS DE ERROR — RESUMEN COMPARATIVO

Métrica	Spline cúbico	Lagrange
RMSE (solicitudes/min)	13.2011	360.4917
MAE (solicitudes/min)	9.4492	150.9138

El spline cúbico presenta un RMSE aproximadamente **27 veces menor** que el de Lagrange (13,20 vs. 360,49 solicitudes/min), confirmando cuantitativamente la superioridad del método por tramos ante datos con nodos irregularmente espaciados y variaciones bruscas de demanda. El error máximo absoluto del spline (26,48 solicitudes/min en 00:00) equivale a un 0,13 % de la escala diaria, mientras que el de Lagrange (1,146,76 solicitudes/min en 24:00) supera en más de seis veces el valor máximo real observado (180 solicitudes/min), evidenciando las oscilaciones espurias propias del fenómeno de Runge en los extremos del intervalo

VI. PROPUESTA DE MEJORA TECNOLÓGICA

El prototipo desarrollado demuestra la viabilidad de la interpolación mediante splines cúbicos para estimar la demanda minuto a minuto. Sin embargo, el sistema opera sobre datos históricos estáticos. La presente sección propone dos mejoras concretas que evolucionan la solución hacia un sistema de despacho adaptativo en tiempo real.

VI-A. Integración con modelo predictivo ARIMA

Como primera mejora se incorporó al prototipo un modelo *Autoregressive Integrated Moving Average* (ARIMA) ligero, que combina la curva interpolada por splines cúbicos con el historial reciente de demanda para generar predicciones a 30 minutos vista. El modelo $ARIMA(p, d, q)$ expresa la demanda futura $\hat{y}_{t+h}$ como función lineal de sus propios valores pasados y de los errores de predicción anteriores:

$$\hat{y}_{t+h} = \mu + \sum_{k=1}^p \phi_k y_{t-k+1} - \sum_{k=1}^q \theta_k \varepsilon_{t-k+1}, \quad (20)$$

donde ϕ_k son los coeficientes autorregresivos, θ_k los coeficientes de media móvil, ε_t el error de predicción en el instante t , y h el horizonte de predicción en minutos. El spline cúbico aporta la tendencia estructural del día (el perfil suave de demanda), mientras que ARIMA captura desviaciones de corto plazo respecto a esa tendencia, tales como picos inesperados por eventos masivos o condiciones climáticas adversas.

Adicionalmente, el sistema emite una alerta automática cuando la demanda estimada supera el umbral crítico de

150 sol/min, notificando al centro de control para que redistribuya vehículos de zonas de baja demanda hacia las zonas afectadas antes de que la congestión de solicitudes se materialice.

VI-B. Propuesta de sensores IoT por localidad en Bogotá

La segunda mejora propone enriquecer el sistema con datos en tiempo real provenientes de dispositivos IoT embebidos en cada taxi de la flota. Concretamente, se plantea instalar un módulo **ESP8266** —microcontrolador Wi-Fi de bajo costo y bajo consumo— en cada vehículo. Este módulo reportaría cada 60 segundos las siguientes variables al servidor central:

- **Geolocalización GPS:** coordenadas del taxi, mapeadas automáticamente a la localidad de Bogotá en la que se encuentra (Usaquén, Chapinero, Suba, Kennedy, etc.).
- **Velocidad instantánea:** derivada del GPS o leída desde el puerto del vehículo.
- **Estado de ocupación:** libre u ocupado, detectado mediante un sensor de presión en el asiento del pasajero o por señal del taxímetro.

Esta arquitectura convierte a OptiTaxi en un sistema *data-driven* completo: los splines cúbicos aportan el perfil estructural de demanda, ARIMA ajusta las predicciones a corto plazo, y los sensores ESP8266 alimentan el modelo con el estado operativo real de la flota, cerrando el ciclo de retroalimentación entre predicción y despacho.

VII. CONCLUSIONES

El presente trabajo demostró que los splines cúbicos constituyen una herramienta matemática robusta y práctica para resolver el problema de interpolación de demanda de transporte en entornos urbanos. A lo largo del desarrollo se obtuvieron las siguientes conclusiones:

En primer lugar, la interpolación por splines cúbicos superó cuantitativamente al polinomio de Lagrange en todos los indicadores de error evaluados. El $RMSE_{LOOCV}$ del spline cúbico fue de 13,20 sol/min, aproximadamente 27 veces menor que el de Lagrange (360,49 sol/min). Esta diferencia se explica por el fenómeno de Runge: el polinomio de grado 10 oscila violentamente en los extremos del intervalo, generando predicciones físicamente absurdas (hasta 1 146 sol/min en el nodo de medianoche) que harían inviable su uso en un sistema de despacho real.

En segundo lugar, la condición de suavidad C^2 garantizada por los splines cúbicos no es un requisito matemático accesorio, sino una necesidad operativa: una curva de demanda con quiebres abruptos en la derivada produciría señales de alerta espurias y reasignaciones innecesarias de la flota, incrementando el consumo de combustible y deteriorando la experiencia del usuario.

En tercer lugar, la validación cruzada LOOCV demostró ser el método de evaluación adecuado para conjuntos de datos pequeños como el presente ($n + 1 = 11$ nodos). Una métrica calculada directamente sobre los nodos de entrenamiento arrojaría error cero por construcción y no aportaría información sobre la capacidad predictiva del modelo.

En cuarto lugar, la implementación en Python con `scipy.interpolate.CubicSpline` permitió evaluar los 1 441 minutos del día en tiempo de cómputo despreciable, confirmando la viabilidad de integrar la interpolación en un sistema web de respuesta en tiempo real basado en Flask.

Finalmente, la propuesta de mejora tecnológica ilustra cómo un resultado matemático relativamente sencillo —la interpolación por splines— puede escalar hacia una solución de ingeniería completa al combinarse con modelos predictivos (ARIMA) y datos en tiempo real provenientes de sensores IoT (ESP8266).

REFERENCIAS

- [1] R. L. Burden y J. D. Faires, *Numerical Analysis*, 10.^a ed. Boston, MA, EE.UU.: Cengage Learning, 2016, pp. 146–198.
- [2] D. Kincaid y W. Cheney, *Numerical Mathematics and Computing*, 6.^a ed. Belmont, CA, EE.UU.: Thomson Brooks/Cole, 2002, pp. 358–401.
- [3] C. de Boor, *A Practical Guide to Splines*. Nueva York, NY, EE.UU.: Springer-Verlag, 1978.
- [4] C. A. Hall y W. W. Meyer, “Optimal error bounds for cubic spline interpolation,” *Journal of Approximation Theory*, vol. 16, no. 2, pp. 105–122, Feb. 1976.
- [5] C. Runge, “Über empirische Funktionen und die Interpolation zwischen äquidistanten Ordinaten,” *Zeitschrift für Mathematik und Physik*, vol. 46, pp. 224–243, 1901.
- [6] J.-P. Berrut y L. N. Trefethen, “Barycentric Lagrange interpolation,” *SIAM Review*, vol. 46, no. 3, pp. 501–517, 2004.
- [7] T. Hastie, R. Tibshirani y J. Friedman, *The Elements of Statistical Learning*, 2.^a ed. Nueva York, NY, EE.UU.: Springer, 2009, pp. 241–249.
- [8] G. E. P. Box, G. M. Jenkins, G. C. Reinsel y G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5.^a ed. Hoboken, NJ, EE.UU.: Wiley, 2015.
- [9] R. J. Hyndman y G. Athanasopoulos, *Forecasting: Principles and Practice*, 2.^a ed. Melbourne, Australia: OTexts, 2018. [En línea]. Disponible: <https://otexts.com/fpp2/>
- [10] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, Sep. 2020.
- [11] P. Virtanen *et al.*, “SciPy 1.0: fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, Feb. 2020.
- [12] J. D. Hunter, “Matplotlib: a 2D graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, May/Jun. 2007.
- [13] M. Grinberg, *Flask Web Development: Developing Web Applications with Python*, 2.^a ed. Sebastopol, CA, EE.UU.: O’Reilly Media, 2018.
- [14] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” tesis doctoral, University of California, Irvine, CA, EE.UU., 2000.
- [15] Espressif Systems, *ESP8266 Technical Reference*, Rev. 1.7. Shanghai, China: Espressif Systems, 2023. [En línea]. Disponible: https://www.espressif.com/sites/default/files/documentation/esp8266-technical_reference_en.pdf
- [16] A. Minteer, *Analytics for the Internet of Things (IoT)*. Birmingham, R.U.: Packt Publishing, 2017, pp. 45–89.
- [17] J. Zhang, F.-Y. Wang y K. Wang, “Data-driven intelligent transportation systems: a survey,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 12, no. 4, pp. 1624–1639, Dic. 2011.
- [18] H. Yao *et al.*, “Deep multi-view spatial-temporal network for taxi demand prediction,” en *Proc. 32nd AAAI Conference on Artificial Intelligence (AAAI)*, Nueva Orleans, LA, EE.UU., 2018, pp. 2588–2595.